BULK IMPORT LEARNING HANDBOOK

# Bulk CSV Product Import

## A detailed code-to-code guide

Master the admin CSV import from browser upload through Cloudinary, Next.js, PostgreSQL, FastAPI, Redis/Celery, SQLAlchemy, and the admin result screen.

**WHAT THIS DOCUMENT COVERS**

- Routes and request/response boundaries
- Code excerpts explained in execution order
- CSV streaming and 300-product transactions
- SKU insert/update rules, progress, failures, notifications

Prepared from the current source in authentication/ and budget-vibe/.

Code excerpts are shortened; use the file map and function names to open the complete code.

# Contents

Follow the import from the browser to the worker, database, and admin UI.


System map 3
Vocabulary and CSV shape 4
1. Admin starts the upload 5
2. Next.js signs the Cloudinary request 6
3. Next.js saves a durable job 7
4. FastAPI enqueues the ID 8
5. Worker claims and streams 9
6. Parser yields 300-product chunks 10
7. Worker matches and uploads images 11
8. One SQLAlchemy transaction per chunk 12
9. SKU determines insert versus update 13
10. Progress and failures 14
11. Status page and notification 15
12. Team-lead walkthrough 16
13. Likely questions 17
14. File-by-file navigation 18
15. Limits and self-test 19
Routes and exact calls 20

# System map

The browser moves file bytes; the jobs worker imports catalog data.

Admin browser -> Next.js signature API -> Cloudinary raw upload
Admin browser -> Next.js initiate API -> PostgreSQL JobRun -> Python FastAPI -> Redis/Celery
Celery worker -> Cloudinary CSV stream + local image folder -> PostgreSQL catalog writes
Admin import page -> Next.js status API -> PostgreSQL
Celery worker -> Next.js internal notification API -> admin notification/socket

Responsibilities:

- Browser: validates/selects one CSV, uploads it directly, starts the job, then opens the status page.
- Next.js: authenticates the admin, signs upload parameters, stores a durable job event, calls jobs, serves status, and publishes completion notifications.
- Cloudinary: stores the original raw CSV and resulting product images.
- FastAPI: authenticates the enqueue request and sends the event ID to Celery.
- Celery worker: streams/parses rows, resolves local image filenames, prepares products, and writes chunks with SQLAlchemy.
- PostgreSQL: shared source of truth. Prisma owns migrations/schema; Python SQLAlchemy maps and writes these existing tables.

# Vocabulary and CSV shape

A CSV row represents a variant; one product can have many rows.

Example:
productName,category,color,size,sku,price,stock,image
Trail Jacket,Outdoor,Black,M,TJ-BLK-M,12000,8,trail-jacket-black.jpg
Trail Jacket,Outdoor,Black,L,TJ-BLK-L,12000,5,trail-jacket-black.jpg

The parser groups adjacent rows with the same exact productName into one product plus its variants. That product group is what counts toward the 300-product chunk. One chunk can contain more than 300 CSV rows.

Required columns: productName, category, color, size, sku, price, stock.
Optional columns: description, image. The image value is a filename, not a URL.

JobRun = one upload/import execution, with CSV URL, queue state, progress, and final outcome.
JobRunItem = one product result in that execution: row index, status, message, resulting productId.
Product, ProductVariant, ProductImage = actual catalog records.

# 1. Admin starts the upload

Read startImport() in the client modal.

```
const signatureResponse = await fetch("/api/admin/upload/csv-signature", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ filename: file.name, sizeBytes: file.size }),
});
```

The modal accepts .csv only and rejects empty files or files over 50 MB. It shows the phases signing, uploading, and starting. It never sends the CSV file bytes to Next.js.

After a job is accepted, the browser navigates to /admin/products/import/{eventId}.

Code: src/components/admin/products/bulk-import-modal.tsx (handleFileChange/startImport).

# 2. Next.js signs the Cloudinary request

The API secret stays on the server; the browser receives a limited signed parameter set.

```
const signature = cloudinary.utils.api_sign_request(
  { folder: CSV_FOLDER, public_id: publicId, timestamp },
  apiSecret,
);
return NextResponse.json({
  cloudName, apiKey, timestamp, folder: CSV_FOLDER,
  publicId, resourceType: "raw", signature,
});
```

POST /api/admin/upload/csv-signature checks the admin session, CSV filename, and size. It generates a random .csv public ID in ecommerce/bulk-import-csv and signs folder, public_id, and timestamp. The API secret is used only to compute the signature; it is never returned.

The browser sends multipart FormData to Cloudinary /raw/upload. Cloudinary returns secure_url, public_id, and version. ‘raw‘ means an untransformed file asset, not an image upload.

Code: src/app/api/admin/upload/csv-signature/route.ts; modal lines that call Cloudinary.

# 3. Next.js saves a durable job

HTTP 202 means accepted for background processing, not finished.

```
const event = await prisma.jobRun.create({
  data: {
    type: "PRODUCT_IMPORT", status: "QUEUED",
    requestedBy: admin.id, cloudinaryPublicId: publicId,
    cloudinarySecureUrl: secureUrl, cloudinaryVersion: version,
    originalFilename: filename, sizeBytes,
    idempotencyKey: publicId, phase: "QUEUED",
  }, select: { id: true },
});
const queued = await sendBulkCsvImportJob({ jobRunId: event.id });
```

POST /api/admin/upload/csv-initiate authenticates the admin, validates metadata and the Cloudinary HTTPS account/path, and creates JobRun in PostgreSQL. Then sendBulkCsvImportJob calls Python. If enqueue fails, Next.js marks the event FAILED and stores the error.

The 202 response includes eventId/jobId/taskId/status. The event ID is the durable reference used by the status page and worker.

Code: src/app/api/admin/upload/csv-initiate/route.ts; src/lib/jobs-client.ts.

# 4. FastAPI enqueues the ID

Redis carries a small message; the worker retrieves the CSV URL from PostgreSQL.

```
result = process_csv_import.apply_async(args=[job.id])
job.celery_task_id = result.id
job.enqueued_at = datetime.utcnow()
job.enqueue_attempts += 1
```

Next.js calls POST /jobs/bulk-upload with the jobs-service Bearer secret. FastAPI verifies the JobRun exists, is a product import, is not already finished, and has a CSV asset URL. It dispatches process_csv_import with the JobRun ID only.

Celery is the background task framework. Redis is its broker here: it holds queued task messages until a worker consumes them. PostgreSQL remains the durable source of job state and input metadata.

Code: budget-vibe/app/main.py, enqueue_bulk_upload(); src/lib/jobs-client.ts.

# 5. Worker claims and streams

The worker reads the remote response directly into the CSV parser.

```
with SessionLocal.begin() as session:
    job = session.scalar(select(JobRunModel).where(
        JobRunModel.id == job_run_id,
        JobRunModel.type == "PRODUCT_IMPORT",
        JobRunModel.status == "QUEUED",
    ).with_for_update(skip_locked=True))
    job.status = "PROCESSING"
    csv_url = job.cloudinary_secure_url
```

process_csv_import claims the event, records PROCESSING/heartbeat/task metadata, and gets the Cloudinary URL. The URL must use HTTPS and the configured Cloudinary account. A bounded HTTP response reader enforces the 50 MB maximum while TextIOWrapper and csv.reader consume data. This avoids holding all CSV rows/products in memory.

Parser checks include required headers, malformed CSV, row shape, productName, and the 10,000 source-row limit.

Code: budget-vibe/app/tasks_bulk_upload.py, _download_csv() and process_csv_import().

# 6. Parser yields 300-product chunks

Products must be complete before the worker commits a batch.

```
for products, chunk_rows in iter_product_chunks(
    text_stream, max_rows=settings.bulk_upload_max_rows, chunk_size=300
):
    prepare_products_and_images(products)
    commit_chunk_with_sqlalchemy(products)
    # continue reading the next chunk
```

iter_product_chunks groups rows by exact productName while reading. Once 300 complete product groups are ready, it yields the batch and its CSV row count. The last batch may be smaller.

All variant rows for a product must be adjacent. If a productName reappears after another product has started, the parser rejects the CSV. This is what lets the worker close a product and chunk without retaining the whole file. The parser keeps the current product, up to 300 product payloads, and a set of names already seen.

Limits: 300 products per chunk, 10,000 CSV data rows, 50 MB file.

Code: budget-vibe/app/bulk_csv.py, iter_product_chunks().

# 7. Worker matches and uploads images

The image column resolves to a local filename on the worker machine.

CSV image filename
-> case-insensitive basename index
-> one safe local file path
-> Cloudinary image upload
-> secure_url/public_id in product payload
-> chunk's prepared-products array

BULK_UPLOAD_IMAGES_DIR points to the image folder visible to Python jobs. The worker recursively indexes files by case-folded basename, ignores symlinks, rejects missing/ambiguous names, and rejects path separators in a CSV filename.

_prepare_product resolves active category/color/size names to database IDs. It uploads distinct images for that product with up to four concurrent upload threads and builds ProductImage and variant data in memory.

Important boundary: Cloudinary image uploads happen before the database transaction. If a later DB write fails, those assets may be orphaned; this importer does not currently delete them.

Code: budget-vibe/app/tasks_bulk_upload.py (_image_index, _resolve_image_path, _upload_image, _prepare_product).

# 8. One SQLAlchemy transaction per chunk

It is one chunk commit, not one SQL statement.

```
with SessionLocal.begin() as session:
    for product in products:
        item = JobRunItemModel(..., status="QUEUED")
        session.add(item)
        try:
            with session.begin_nested():  # product savepoint
                product_id, action = import_product(session, payload)
            item.status = "COMPLETED"
            item.message = f"{action} {product['name']}"
        except Exception as exc:
            item.status = "FAILED"
            item.message = str(exc)
    session.flush()
    update_progress(session, job_id)
# exiting the context commits this chunk
```

The outer SessionLocal.begin() transaction includes catalog writes, JobRunItems, and progress. ‘session.begin_nested()‘ creates a savepoint per product: a bad product can roll back independently while the worker records its failure and continues with other products.

SQLAlchemy still sends multiple SQL statements for Product, ProductVariant, ProductImage, JobRunItem, and progress. They commit together once for the chunk. If the outer commit fails, that chunk rolls back. Earlier committed chunks stay committed.

SessionLocal uses autoflush=False, so code calls flush before reading counts that include current transaction changes.

Code: budget-vibe/app/tasks_bulk_upload.py; app/db.py.

# 9. SKU determines insert versus update

productName groups CSV rows; SKU matches catalog variants.

```
existing = session.scalars(
    select(ProductVariantModel).where(
        ProductVariantModel.sku.in_(skus)
    )
).all()
```

```
if a supplied SKU matches one product:
    update product fields and matching variants
    add any new SKUs to that product
    replace its database image rows
else:
    create Product, then insert images and variants
```

Incoming SKUs are normalized/validated. If no SKU matches, the helper creates a Product with a unique slug, flushes the parent row, then inserts child image/variant rows. If supplied SKUs belong to one existing product, it updates that product and the matching variants; new SKUs are added. If the supplied SKUs belong to multiple products, this CSV product fails.

A repeat of the same full CSV updates by SKU. Variants omitted from a later partial CSV are not deleted. Database image rows are updated/reused by position, but reimported files create new Cloudinary assets, so old assets can remain unreferenced.

Code: budget-vibe/app/product_import_orm.py (_validated_values, import_product).

# 10. Progress and failures

Each committed chunk is a durable checkpoint.

JobRun: totalRows, totalItems, processedItems, successCount, failedCount
JobRunItem: rowIndex, status, message, productId

PROCESSING -> COMPLETED | PARTIAL | FAILED

After a chunk, the worker recounts JobRunItem statuses, updates progress and heartbeat, and commits. At end of file, finalize_job_progress sets terminal status and summary.

COMPLETED means every product succeeded. PARTIAL means some products succeeded and some failed, or a fatal error occurred after earlier committed products. FAILED means no products succeeded or the job could not be processed. Each item message says Created/Updated or gives a failure reason. The admin UI displays Created as Inserted.

A malformed later section stops later chunks; already committed chunks remain. A failed product savepoint does not discard valid products in that same chunk. A failure of the outer chunk transaction rolls back the entire current chunk.

# 11. Status page and notification

Progress polling and socket notifications are two separate mechanisms.

GET /api/admin/jobs/{id}
  -> Prisma reads JobRun + JobRunItem in PostgreSQL
  -> import page polls about every 3 seconds
  -> progress and inserted/updated/failed outcomes render

At terminal state:
Celery -> POST /api/internal/socket-notify
    -> upsert AdminNotification
    -> publish socket update to admin notification UI

The import page loads initial state with Prisma and then polls the Next.js status route. It does not call FastAPI for every refresh because both apps share PostgreSQL. The worker callback runs only after terminal status. If that callback/socket fails, the DB status remains available to the polling page.

Inserted/updated counts are derived from each completed JobRunItem.message; the item links to its product. The notification socket updates the admin notification UI; it is not the source of chunk progress.

Code: src/app/api/admin/jobs/[id]/route.ts; import-job-status.tsx; app/next_notify.py; src/app/api/internal/socket-notify/route.ts.

# 12. Team-lead walkthrough

You can use this as your oral explanation.

The browser validates the CSV and asks Next.js for a signed raw-upload authorization. It uploads the file directly to Cloudinary, so Next.js never proxies the CSV bytes. The browser then sends Cloudinary's secure URL and metadata to Next.js.

Next.js authenticates the admin, stores a JobRun in PostgreSQL, and calls the Python jobs API. FastAPI authenticates that request and enqueues only the JobRun ID in Celery/Redis. The worker reads the URL from the database, claims the event, and streams CSV rows.

Rows for each product are grouped into variants. The worker resolves image filenames against its local image folder, uploads those images, and prepares at most 300 complete products. SQLAlchemy writes each batch in one outer transaction, with savepoints per product. It records outcomes and progress, commits, then reads the next chunk.

When processing ends, the worker finalizes the JobRun and calls Next.js to upsert/publish an admin notification. The import page reads progress from the shared PostgreSQL database through Prisma.

# 13. Likely questions

Answer these from the code, not from memory alone.

Why not send the CSV through Next.js?
That would make the web server receive/buffer large file bytes. The browser uploads directly to Cloudinary.

Why create JobRun before enqueueing?
The queue message carries only its ID. The worker needs a durable row containing the Cloudinary URL and progress.

What does 300 count?
Complete grouped products, not CSV rows. A product may have several variant rows.

What is atomic?
The database writes for one chunk. Cloudinary uploads are external side effects and earlier chunks have already committed.

How does a repeat upload update instead of insert?
The SQLAlchemy helper looks up variant SKUs. Existing SKUs map to their product; matching data is updated and new SKUs may be added.

Why Prisma and SQLAlchemy together?
Prisma owns the schema/migrations. Python SQLAlchemy maps the same tables and writes CSV import data. SQLAlchemy does not create these tables.

Why keep JobRunItem?
It gives an admin per-product status/message/product link and lets the job compute accurate progress.

# 14. File-by-file navigation

Open these files in this order while rehearsing the flow.

NEXT.JS
src/components/admin/products/bulk-import-trigger.tsx - opens the modal
src/components/admin/products/bulk-import-modal.tsx - validates, signs, uploads, starts, navigates
src/app/api/admin/upload/csv-signature/route.ts - authenticated Cloudinary signature
src/app/api/admin/upload/csv-initiate/route.ts - validates metadata, creates JobRun, calls jobs
src/lib/jobs-client.ts - authenticated jobs-service client
src/app/admin/(dashboard)/products/import/[jobId]/page.tsx - initial Prisma query
src/app/api/admin/jobs/[id]/route.ts - polling response
src/components/admin/products/import-job-status.tsx - progress and outcomes
src/app/api/internal/socket-notify/route.ts - notification upsert/socket publish

PYTHON JOBS
app/main.py - authenticated /jobs/bulk-upload endpoint
app/tasks_bulk_upload.py - worker claim, stream, images, chunk transaction
app/bulk_csv.py - CSV validation and 300-product generator
app/product_import_orm.py - SKU upsert and final status
app/models.py - SQLAlchemy mappings for Prisma-owned tables
app/db.py - SQLAlchemy engine/session
app/next_notify.py - terminal callback to Next.js

Do not confuse the CSV flow with the separate legacy JSON endpoint ‘/jobs/products/import‘ in app/main.py. The admin CSV modal calls ‘/jobs/bulk-upload‘; the JSON endpoint queues one task per product. app/job_store.py also uses raw SQL for other/legacy jobs; the CSV catalog writes explained here use SQLAlchemy ORM.

# 15. Limits and self-test

Master the data shape, then narrate the asynchronous boundaries.

Current limits and rules:

- CSV file: 50 MB maximum.
- CSV rows: 10,000 maximum.
- Product groups per DB transaction: at most 300.
- Product’s variant rows must be adjacent and share exact productName.
- Existing product matching: SKU, not productName.
- Local image files must be present where the Python worker can read them.
- A chunk uses one outer transaction; it still consists of many SQL statements.
- Progress comes from PostgreSQL polling; terminal admin notifications use a callback/socket.
- A full repeat import updates matching SKUs; omitted variants are not deleted.

Self-test:
1. Where do file bytes travel?
2. Where is the Cloudinary API secret used?
3. What is inside the Celery message?
4. What exactly is limited to 300?
5. What identifies an existing product?
6. What rolls back if one product savepoint fails?
7. Why can status still be read if socket notification fails?
8. Which endpoint is the CSV path, and which is the separate JSON path?

# Routes and exact calls

Walk this list in order to explain who calls whom, what crosses each boundary, and what comes back.

1. Browser -> Next.js: POST /api/admin/upload/csv-signature. Request: filename and sizeBytes. Response: Cloudinary cloudName, apiKey, timestamp, folder, publicId, resourceType, signature. Requires admin session.

2. Browser -> Cloudinary: POST https://api.cloudinary.com/v1_1/{cloudName}/raw/upload. Multipart fields include the CSV file and signed parameters. Response contains secure_url, public_id, version. Next.js is not in this file-byte path.

3. Browser -> Next.js: POST /api/admin/upload/csv-initiate. Request: secureUrl, publicId, version, filename, sizeBytes. Next validates the Cloudinary URL, inserts JobRun, calls Python, and returns HTTP 202 with eventId/jobId/taskId/status.

4. Next.js -> Python FastAPI: POST {FASTAPI_JOBS_URL}/jobs/bulk-upload. Header: Authorization: Bearer {JOBS_API_SECRET}. JSON: {"jobRunId":"..."}. FastAPI returns 202 with task/job details.

5. FastAPI -> Celery/Redis: process_csv_import.apply_async(args=[job.id]). This is a task dispatch, not an HTTP call. Celery message contains the durable JobRun ID; worker loads the URL from PostgreSQL.

6. Admin browser -> Next.js: GET /api/admin/jobs/{id}, about every 3 seconds. Authenticated Next route reads JobRun and JobRunItems using Prisma from shared PostgreSQL.

7. Python worker -> Next.js on terminal status: POST {APP_URL}/api/internal/socket-notify, Bearer jobs secret, JSON {"jobRunId":"..."}. Next upserts AdminNotification and publishes an admin socket update. This is a completion notification; polling is the status/progress source.

Health check: the review UI can call GET /api/admin/jobs/health, which checks GET {FASTAPI_JOBS_URL}/health before starting an import.

> The separate JSON endpoint POST /jobs/products/import is not called by this CSV modal. It is a different path with one Celery task per product; do not mix it up with POST /jobs/bulk-upload and the 300-product CSV chunk worker.